

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Выполнили:

Бардышев Артём Антонович,
студент группы N3346

(подпись)

Проверил:

Ищенко Алексей Петрович,
преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение.....	4
1 sys_installer.py (sys_doc.exe).....	5
1.1 Алгоритм работы:.....	5
1.1.1 Сбор системной информации.....	5
1.1.2 Кодирование данных	5
1.1.3 Генерация криптографических ключей	5
1.1.4 Создание цифровой подписи.....	5
1.1.5 Настройка системы	5
1.2 Технические детали реализации	6
2 secur.py (secur.exe).....	7
2.1 Алгоритм работы.....	7
2.1.1 Запрос секретного ключа.....	7
2.1.2 Чтение подписи из реестра.....	7
2.1.3 Загрузка публичного ключа.....	7
2.1.4 Проверка подписи.....	7
2.1.5 Декодирование и отображение.....	7
3 Технологии и библиотеки	9
4 Удаленный сбор информации о системе (3b).....	10
4.1 server.py (server_gui.exe).....	10
4.2 client.py (virus_detect.exe).....	11
Заключение.....	15

ВВЕДЕНИЕ

Цель работы – Разработка системы сбора и защиты информации о системе с использованием цифровой подписи и удаленного сбора данных. Изучение механизмов криптографической защиты целостности данных и создание клиент-серверных приложений для удаленного мониторинга систем.

Цифровая подпись — это криптографический механизм, обеспечивающий:

- Аутентификацию — подтверждение авторства данных
- Целостность — гарантию неизменности данных
- Неотрекаемость — невозможность отказаться от авторства

В данной работе используется алгоритм RSA с хэш-функцией SHA-256 и схемой подписи PKCS1v15.

Архитектура системы

Система состоит из двух независимых подсистем:

1. Локальная система защиты (3a) — сбор и защита информации на одном компьютере
2. Удаленная система сбора (3b) — клиент-серверная архитектура для сбора данных с удаленных машин

```
sys_installer.py (sys_doc.exe)
sys_installer.py (sys_doc.exe)
sys_installer.py (sys_doc.exe)
```

1 SYS_INSTALLER.PY (SYS_DOC.EXE)

Назначение: Программа установки, сбора информации и генерации защищенных данных.

1.1 Алгоритм работы:

1.1.1 Сбор системной информации

- Имя пользователя (``os.environ.get("USERNAME")``)
- Имя компьютера (``platform.node()``)
- Версия операционной системы (``platform.platform()``)
- Модель процессора (через ``wmic cpu get Name``)
- Общий объем оперативной памяти (через ``wmic ComputerSystem get TotalPhysicalMemory``)
- Временная метка сбора данных (UTC)

1.1.2 Кодирование данных

- Информация преобразуется в JSON-формат
- JSON кодируется в base64 для безопасного хранения
- Результат сохраняется в файл ``sys.tat``

1.1.3 Генерация криптографических ключей

- Генерируется пара RSA-ключей (2048 бит, публичная экспонента 65537)
- Приватный ключ используется для подписи
- Публичный ключ экспортируется в формат PEM и сохраняется в ``pubkey.pem``

1.1.4 Создание цифровой подписи

- Содержимое файла ``sys.tat`` подписывается приватным ключом
- Используется схема PKCS1v15 с хэш-функцией SHA-256
- Подпись кодируется в base64 и сохраняется в реестре Windows

1.1.5 Настройка системы

- Создается папка ``data`` на рабочем столе
- Сохраняются файлы: ``sys.tat``, ``pubkey.pem``, ``secur.exe`` (или ``secur.py``)

- Создается ассоциация файлов `.tat` с программой просмотра через реестр Windows
- При двойном клике на `.tat` файл автоматически открывается через `secur.exe`

1.2 Технические детали реализации

Пример генерации ключей:

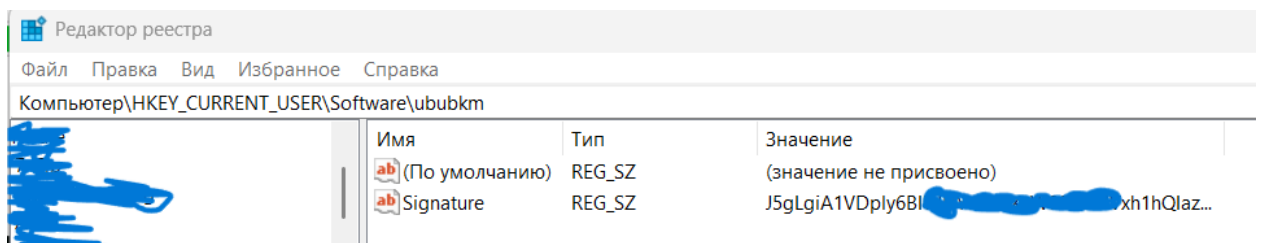
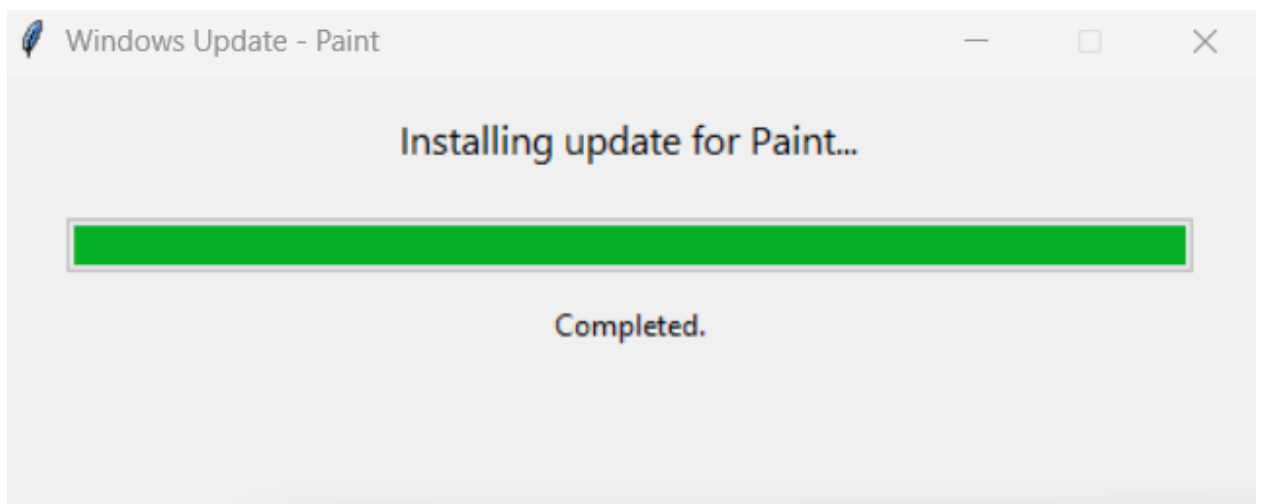
```
priv, pub = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
```

Пример создания подписи

```
signature = priv.sign(
    encoded.encode('utf-8'),
    padding.PKCS1v15(),
    hashes.SHA256()
)
```

Интерфейс программы:

Программа имеет графический интерфейс, имитирующий установку обновления Windows Paint, что делает её менее заметной для пользователя.



2 SECUR.PY (SECUR.EXE)

Программа для проверки целостности и просмотра защищенной информации.

2.1 Алгоритм работы

2.1.1 Запрос секретного ключа

- Программа запрашивает у пользователя номер секции реестра
- Ожидается значение "3442" (секретный ключ доступа)
- При неверном значении доступ блокируется

2.1.2 Чтение подписи из реестра

- Открывается раздел `HKCU\Software\ububkm`
- Читается значение `Signature`
- Подпись декодируется из base64

2.1.3 Загрузка публичного ключа

- Ищется файл `pubkey.pem` в той же папке, что и `sys.tat`
- Публичный ключ загружается из PEM-формата

2.1.4 Проверка подписи

- Содержимое файла `sys.tat` считывается
- Выполняется проверка подписи с использованием публичного ключа
- Используется алгоритм RSA-PKCS1v15 с SHA-256

2.1.5 Декодирование и отображение

- При успешной проверке файл декодируется из base64
- JSON-данные парсятся и отображаются в читаемом виде
- Пользователь видит информацию о системе

```
C:\Users\ububkm\Desktop\da  ×  +  ∨  
Enter registry section number with the digital signature (hint: a number):  
> 5  
  
=== SYSTEM INFORMATION (decoded) ===  
user: [REDACTED]  
computer: [REDACTED]  
os: [REDACTED]  
cpu: [REDACTED]  
total_memory_bytes: [REDACTED]  
timestamp_utc: [REDACTED]  
=====
```

3 ТЕХНОЛОГИИ И БИБЛИОТЕКИ

Используемые технологии:

- Python 3.9+ — язык программирования
- cryptography — библиотека для криптографических операций
- ``rsa.generate_private_key()`` — генерация RSA-ключей
- ``serialization`` — работа с форматами ключей (PEM)
- ``padding.PKCS1v15()`` — схема подписи
- ``hashes.SHA256()`` — хэш-функция
- PyInstaller — упаковка в исполняемые файлы
- Windows Registry API (``winreg``) — работа с реестром
- tkinter — графический интерфейс
- base64 — кодирование данных
- json — формат данных
- platform, os, subprocess — сбор системной информации

Команда сборки:

```
pyinstaller --onefile --noconsole --name sys_doc sys_installer.py  
pyinstaller --onefile --console --name secur secur.py
```


4 УДАЛЕННЫЙ СБОР ИНФОРМАЦИИ О СИСТЕМЕ (ЗВ)

Создание клиент-серверной системы для удаленного сбора информации о системе. Клиент маскируется под легитимную программу безопасности, собирает данные и отправляет их на сервер.

Архитектура решения:

4.1 server.py (server_gui.exe)

Назначение: HTTP-сервер для приема и сохранения системной информации от клиентов.

Архитектура:

- Flask-сервер на порту 8443
- GUI-интерфейс для визуализации процесса
- Многопоточность — сервер работает в отдельном потоке, GUI в основном

Алгоритм работы:

1. Инициализация сервера:

- Создается папка `data` на рабочем столе (если не существует)
- Запускается Flask-сервер на `0.0.0.0:8443` (принимает подключения со всех интерфейсов)

- Запускается GUI-интерфейс

2. Обработка запросов:

- Сервер слушает POST-запросы на `/collect`
- Проверяется API-ключ в заголовке или теле запроса
- При неверном ключе возвращается ошибка 401

3. Сохранение данных:

- Извлекается информация о клиенте (IP, hostname, username)
- Создается уникальный идентификатор клиента: `{IP}\_{hostname}\_{username}`
- Создается подпапка для каждого клиента в `data`
- Данные сохраняются в JSON-файл с временной меткой: `YYYYMMDDTHHMMSSZ.json`

4. Уведомление GUI:

- Через очередь событий (`queue.Queue`) отправляется уведомление в GUI
- GUI обновляется, показывая успешное получение данных

Структура сохраняемых данных:

```
{
  "received_at_utc": "2024-12-24T18:30:00Z",
  "client_ip": "192.168.1.100",
  "info": {
    "user": "username",
    "computer": "DESKTOP-ABC123",
    "os": "Windows-10-10.0.19041",
    "memory_total_gb": 16.0,
    "memory_used_gb": 8.5,
    "cpu_model": "Intel Core i7-9700K",
    "disks": [...],
    "nics": {...}
  }
}
```

4.2 client.py (virus_detect.exe)

Назначение: Клиентское приложение для сбора и отправки системной информации на сервер.

Маскировка:

Программа маскируется под легитимную программу безопасности "System Security Check", что снижает подозрения пользователя.

Алгоритм работы:

1. Инициализация GUI:

- Отображается окно "System Security Check"
- Показывается анимированный прогресс-бар
- Текст "Performing system security check..."

2. Сбор системной информации (в фоновом потоке):

Основная информация:

- Имя пользователя (``getpass.getuser()``)
- Имя компьютера (``socket.gethostname()``)
- Версия ОС (``platform.platform()``)

Информация о памяти:

- Общий объем (``psutil.virtual_memory().total``)
- Используемый объем (``psutil.virtual_memory().used``)
- Процент использования (``psutil.virtual_memory().percent``)

Информация о процессоре:

- Модель (``platform.processor()``)
- Количество физических ядер (``psutil.cpu_count(logical=False)``)
- Количество логических ядер (``psutil.cpu_count(logical=True)``)

Информация о дисках:

- Для каждого раздела:
- Устройство (`psutil.disk_partitions()`)
- Тип файловой системы
- Общий объем, использованный объем, процент использования

Сетевые интерфейсы:

- Для каждого интерфейса:
- IPv4 адрес
- MAC-адрес

Время работы системы:

- Uptime в секундах (`psutil.boot_time()`)

3. Имитация проверки безопасности:

- Программа "работает" 5 секунд (имитация сканирования)
- Показывается прогресс-бар

4. Отправка данных на сервер:

- Формируется JSON-запрос с API-ключом и собранной информацией
- Отправляется POST-запрос на `http://[SERVER_IP]:8443/collect`
- Таймаут запроса: 10 секунд

5. Отображение результата:

- После завершения показывается "System check complete"
- Текст "No threats found. Report sent successfully." (при успехе)
- Или сообщение об ошибке (при неудаче)

Конфигурация:

Перед сборкой необходимо указать IP-адрес сервера в коде:

```
SERVER_URL = "http://192.168.1.50:8443/collect"    IP сервера
API_KEY = "b7f9d2a3-4e6c-41f2-9a8e-5c3d7f0b2e1c"  Должен совпадать с сервером
```

Технологии и библиотеки

Используемые технологии:

- Python 3.9+ — язык программирования
- Flask — веб-фреймворк для сервера
- psutil — библиотека для сбора системной информации
- requests — библиотека для HTTP-запросов
- tkinter — графический интерфейс
- threading — многопоточность

- queue — межпоточная коммуникация
- datetime — работа с временными метками
- socket — получение сетевой информации
- getpass — получение имени пользователя
- PyInstaller — упаковка в исполняемые файлы

Установка зависимостей:

```
pip install flask psutil requests
```

Команды сборки:

Сервер

```
pyinstaller --onefile --noconsole --name server_gui server.py
```

Клиент

```
pyinstaller --onefile --noconsole --name virus_detect client.py
```

Сетевая конфигурация

Требования:

- Оба компьютера должны находиться в одной локальной сети
- На сервере должен быть открыт порт 8443 (или настроен firewall)
- Необходимо знать локальный IP-адрес сервера

Определение IP-адреса сервера:

Windows

```
ipconfig
```

Linux/Mac

```
ifconfig
```

или

```
ip addr
```

Результаты работы

Достигнутые цели

1. Реализована система защиты информации:

- Сбор системной информации
- Генерация RSA-ключей (2048 бит)
- Создание цифровой подписи (RSA-PKCS1v15 + SHA-256)
- Хранение подписи в реестре Windows
- Проверка целостности данных

2. Реализована система удаленного сбора данных:

- HTTP-сервер на Flask
- Клиентское приложение с маскировкой
- Сбор расширенной системной информации
- Организованное хранение данных по клиентам

3. Созданы пользовательские интерфейсы:

- Графические интерфейсы для обеих подсистем
- Интуитивно понятное взаимодействие
- Визуальная обратная связь о процессе работы

4. Обеспечена безопасность:

- Криптографическая защита данных
- Аутентификация через секретный ключ
- Проверка API-ключа для сервера
- Защита от несанкционированного доступа

Статистика реализации

- Строк кода: ~600 (3a) + ~400 (3b)
- Используемых библиотек: 8
- Созданных исполняемых файлов: 4
- Поддерживаемых форматов данных: JSON, PEM, base64
- Собираемых параметров системы: 15+

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены и реализованы:

Теоретические знания

1. Криптография:

- Принципы работы цифровой подписи
- Алгоритм RSA и его применение
- Хэш-функции (SHA-256)
- Схемы подписи (PKCS1v15)

2. Системное программирование:

- Работа с реестром Windows
- Сбор системной информации через API
- Работа с файловой системой
- Ассоциация файлов

3. Сетевое программирование:

- Создание HTTP-серверов
- Клиент-серверная архитектура
- Обработка HTTP-запросов
- Сетевая безопасность

Практические навыки

1. Разработка защищенных приложений:

- Использование криптографических библиотек
- Реализация механизмов аутентификации
- Защита целостности данных

2. Создание пользовательских интерфейсов:

- Работа с tkinter
- Многопоточность в GUI-приложениях
- Обработка событий

3. Упаковка и распространение:

- Использование PyInstaller
- Создание автономных исполняемых файлов
- Настройка параметров сборки